

Práctica 2F – Remediación de Vulnerabilidades OWASP

SEGURIDAD EN EL DESARROLLO DE APLICACIONES

Alumna: Luz Celeste López Rivera

Docente: Gerardo Ramírez Villareal

Objetivo

Aplicar controles de seguridad para corregir las vulnerabilidades identificadas en las prácticas anteriores, verificando que los ataques realizados previamente ya no puedan ejecutarse dentro de la aplicación VulnerableApp.

Desarrollo

En esta práctica se creó una rama denominada `secure`, en la cual se implementaron distintas medidas de seguridad para mitigar las vulnerabilidades detectadas en las actividades anteriores.

Corrección de SQL Injection

Se reemplazaron las consultas construidas mediante concatenación de cadenas por consultas realizadas con Entity Framework y LINQ.

Anteriormente era posible manipular la búsqueda utilizando entradas maliciosas como: ' OR '1'='1

Después de la corrección, la búsqueda únicamente devuelve resultados válidos y ya no interpreta instrucciones SQL ingresadas por el usuario.

Corrección de autenticación insegura

Se eliminaron las credenciales hardcodeadas y las consultas SQL vulnerables utilizadas durante el inicio de sesión.

La autenticación pasó a realizarse mediante consultas seguras utilizando Entity Framework y se incorporó la validación de contraseñas mediante BCrypt para evitar comparaciones directas de texto plano.

Protección de contraseñas

Se instaló la librería BCrypt.Net-Next y se generaron hashes para las contraseñas almacenadas en la base de datos.

Las contraseñas dejaron de almacenarse en texto plano y fueron sustituidas por cadenas cifradas generadas mediante BCrypt.

Posteriormente el proceso de autenticación fue modificado para validar las credenciales utilizando:

```
BCrypt.Net.BCrypt.Verify()
```

Corrección de XSS

Se eliminó el uso de: `@Html.Raw(comment)` que permitía la ejecución de código JavaScript dentro del navegador.

La aplicación fue modificada para mostrar el contenido como texto, evitando la ejecución de scripts maliciosos ingresados por los usuarios.

Corrección de IDOR

Se agregaron validaciones de sesión y autorización dentro del `ApiController`, ahora la aplicación verifica la identidad del usuario antes de permitir el acceso a la información solicitada y también se eliminaron campos sensibles como las contraseñas de las respuestas enviadas por la API.

Resultados obtenidos

Después de aplicar las correcciones se verificó que:

- Los intentos de SQL Injection ya no modifican el comportamiento de la búsqueda.
- Las credenciales inseguras dejaron de funcionar.
- Las contraseñas ya no se almacenan en texto plano.
- Los scripts XSS ya no se ejecutan en el navegador.
- Los endpoints de la API ya no exponen información sensible sin validación.

Tabla comparativa final

Vulnerabilidad	Antes	Después
SQL Injection	Permitía manipular consultas	Consultas seguras mediante LINQ
Autenticación insegura	Credenciales hardcodeadas y SQL vulnerable	Validación mediante Entity Framework y BCrypt
Contraseñas	Almacenadas en texto plano	Almacenadas como hash BCrypt
XSS	Ejecutaba código JavaScript	El contenido se muestra como texto
IDOR	Permitía consultar usuarios por ID	Acceso restringido mediante validaciones

Problemas encontrados

Encontré algunos inconvenientes relacionados con la instalación de la librería `BCrypt`, la actualización de las contraseñas almacenadas en la base de datos y la validación de sesiones dentro del `ApiController`. Sin embargo, estos problemas fueron corregidos mediante ajustes en el código y pruebas adicionales de funcionamiento.

Conclusión

La práctica me hizo comprender la importancia de aplicar controles de seguridad desde el desarrollo de aplicaciones. Con la implementación de mecanismos de validación, autorización, almacenamiento seguro de contraseñas y protección contra inyección de código, fue posible reducir significativamente los riesgos presentes en la aplicación vulnerable, todas las vulnerabilidades identificadas durante las prácticas anteriores fueron mitigadas y verificadas mediante pruebas de funcionamiento.